

KANTANOVA

API Reference & System Architecture

For Frontend Integration
Version 1.0 | Base URL: /api/v1

1. System Overview

Kantanova is a multi-sided e-commerce and logistics platform that digitalises Ghana's largest physical fashion market. It connects three user types on a mobile-first platform.

Platform	Kantanova Digital Marketplace
Base URL	https://api.kantanova.com/api/v1 (production)
Dev URL	http://localhost:5000/api/v1
Auth method	JWT via httpOnly cookies (access_token + refresh_token)
Currency	GHS (Ghana Cedis)
Versioning	URI-based — all endpoints prefixed with /v1

2. User Roles

Every authenticated request carries a role in the JWT payload. The role determines which endpoints are accessible.

Role	Status flow	Description
CUSTOMER	ACTIVE on register	Browses products, places orders, tracks deliveries, confirms receipt
VENDOR	PENDING → ACTIVE	Lists products, receives orders, manages inventory, withdraws earnings
RIDER	PENDING → ACTIVE	Accepts deliveries, updates delivery status, withdraws earnings
ADMIN	Always ACTIVE	Full platform control — verifies vendors/riders, manages orders and disputes

3. Authentication

All auth tokens are delivered and expected as httpOnly cookies. The frontend must send requests with credentials: true (fetch) or withCredentials: true (axios).

3.1 Cookie Reference

Cookie	Lifetime	Purpose
access_token	15 minutes	Sent with every protected request automatically
refresh_token	7 days	Used only to get a new access_token via POST /auth/refresh

3.2 Endpoints

Method	Endpoint	Access	Description
POST	/auth/register/customer	Public	Register a new customer account
POST	/auth/register/vendor	Public	Register a new vendor — starts as PENDING_VERIFICATION
POST	/auth/register/rider	Public	Register a new rider — starts as PENDING_VERIFICATION
POST	/auth/login	Public	Login with phone + password. Sets both cookies
POST	/auth/refresh	Public	Exchange refresh_token cookie for new token pair
POST	/auth/logout	Auth	Clears both cookies and invalidates refresh token
GET	/auth/me	Auth	Returns the current authenticated user from JWT
POST	/auth/admin/create-user	Admin	Admin creates any user with any role

3.3 Register Customer — Request Body

```
{
  "fullName": "Kwame Mensah",
  "phone": "0551234567",
  "email": "kwame@email.com", // optional
  "password": "SecurePass123"
}
```

3.4 Register Vendor — Request Body

```
{
  "fullName": "Akosua Boateng",
  "phone": "0241234567",
  "email": "akosua@shop.com", // optional
  "password": "SecurePass123",
  "shopName": "Akosua Fashion",
  "stallNumber": "B-42",
  "momoNumber": "0241234567"
}
```

3.5 Register Rider — Request Body

```
{
```

```
"fullName": "Kofi Rider",
"phone": "0271234567",
"password": "SecurePass123",
"vehicleType": "MOTORCYCLE", // MOTORCYCLE | BICYCLE | CAR | ON_FOOT
"vehiclePlate": "GR-1234-22" // optional
}
```

3.6 Login — Request Body

```
{
  "phone": "0551234567",
  "password": "SecurePass123"
}
```

4. Users

Manages profiles for all three user types. Vendors and riders have extended profiles linked to the base user account.

4.1 Endpoints

Method	Endpoint	Access	Description
GET	/users/profile	Auth	Get the current user's full profile
PATCH	/users/profile	Auth	Update name, email, profile photo, FCM token
GET	/users/vendors/:id	Auth	Get a vendor's public shop profile
PATCH	/users/vendors/profile	Vendor	Update own shop name, description, banner, MoMo number
GET	/users/vendors/wallet	Vendor	Get wallet balance
PATCH	/users/riders/profile	Rider	Update vehicle type and plate
PATCH	/users/riders/location	Rider	Push current GPS coordinates
PATCH	/users/riders/status	Rider	Toggle AVAILABLE / OFFLINE / ON_DELIVERY
GET	/users/riders/wallet	Rider	Get wallet balance
GET	/users/admin/all	Admin	List all users — filter by role, status, search
GET	/users/admin/pending-verifications	Admin	Get all unverified vendors and riders
GET	/users/admin/:id	Admin	Get any user by UUID
PATCH	/users/admin/:id/suspend	Admin	Suspend a user account
PATCH	/users/admin/:id/activate	Admin	Reactivate a suspended account
PATCH	/users/admin/vendors/:id/verify	Admin	Verify a vendor — sets isVerified + ACTIVE status
PATCH	/users/admin/riders/:id/verify	Admin	Verify a rider — allows them to go online

4.2 Update Profile — Request Body

```
{
  "fullName": "Kwame Mensah", // optional
  "email": "new@email.com", // optional
  "profilePhoto": "https://res.cloudinary.com/...", // optional
  "fcmToken": "firebase-device-token" // optional
}
```

4.3 Update Rider Location — Request Body

```
{
  "lat": 5.5557,
  "lng": -0.1969
}
```

4.4 Update Rider Status — Request Body

```
{  
  "status": "AVAILABLE"    // AVAILABLE | OFFLINE | ON_DELIVERY  
}
```

5. Catalog

Covers product categories, product listings with variants, and customer reviews. Only verified vendors can create products.

5.1 Endpoints

Method	Endpoint	Access	Description
GET	/catalog/categories	Public	List all active product categories
POST	/catalog/categories	Admin	Create a new category
GET	/catalog/products	Public	Browse products — filter by category, vendor, price, search, sort
GET	/catalog/products/:id	Public	Get full product details including variants
POST	/catalog/products	Vendor	Create a new product listing
GET	/catalog/vendor/products	Vendor	Get own product listings
PATCH	/catalog/products/:id	Vendor	Update own product details or variants
DELETE	/catalog/products/:id	Vendor	Delete own product
GET	/catalog/products/:id/reviews	Public	Get all reviews for a product
POST	/catalog/reviews	Customer	Leave a review — one per product per customer

5.2 Create Product — Request Body

```
{
  "name": "Nike Air Force 1",
  "description": "Classic white leather sneakers",
  "basePrice": 350.00,
  "categoryId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "images": [
    "https://res.cloudinary.com/kantanova/image/upload/v1/products/img1.jpg"
  ],
  "variants": [
    { "size": "42", "color": "White", "material": "Leather", "price": 350.00, "stock": 8 },
    { "size": "43", "color": "White", "material": "Leather", "price": 370.00, "stock": 4 },
    { "size": "42", "color": "Black", "material": "Leather", "price": 350.00, "stock": 6 }
  ]
}
```

5.3 Query Parameters for GET /catalog/products

Param	Example	Description
categoryId	uuid	Filter by category
vendorId	uuid	Filter by vendor shop
search	sneaker	Search by product name
minPrice	100	Minimum price filter (GHS)

Param	Example	Description
maxPrice	500	Maximum price filter (GHS)
sortBy	popular	newest popular rating price_asc price_desc
page	1	Page number (default 1)
limit	20	Items per page (default 20)

6. Orders

Handles the full order lifecycle from cart through checkout, dispatch, delivery, and completion. The state machine enforces valid transitions only.

6.1 Order Status State Machine

Status	Who sets it	Meaning
PENDING	System (on checkout)	Order created, awaiting payment
PAID	Payment webhook (Hubtel)	Payment confirmed, vendor alerted
DISPATCHED	Vendor / Admin	Vendor packed order, rider assigned and heading over
EN_ROUTE	Rider	Rider picked up order, heading to customer
DELIVERED	Rider	Rider dropped off order, awaiting customer confirmation
COMPLETED	Customer	Customer confirmed receipt — wallets credited
CANCELLED	Customer / Admin	Cancelled before dispatch — stock restored
REFUNDED	Admin	Refund issued after delivery

6.2 Endpoints

Method	Endpoint	Access	Description
GET	/orders/cart	Customer	Get current cart with totals
POST	/orders/cart	Customer	Add item to cart
PATCH	/orders/cart/:itemId	Customer	Update cart item quantity
DELETE	/orders/cart/:itemId	Customer	Remove item from cart
DELETE	/orders/cart	Customer	Clear entire cart
POST	/orders/checkout	Customer	Convert cart to order — deducts stock
GET	/orders/my-orders	Customer	Get own order history
GET	/orders/my-orders/:id	Customer	Get single order details
PATCH	/orders/my-orders/:id/cancel	Customer	Cancel a PENDING or PAID order
PATCH	/orders/my-orders/:id/confirm-delivery	Customer	Confirm receipt — triggers wallet credit
GET	/orders/vendor-orders	Vendor	Get all orders containing vendor's products
PATCH	/orders/vendor/:id/status	Vendor	Mark order as DISPATCHED (ready for pickup)
GET	/orders/rider/orders	Rider	Get all assigned deliveries
GET	/orders/rider/active	Rider	Get current active delivery
PATCH	/orders/rider/:id/status	Rider	Update to EN_ROUTE or DELIVERED
GET	/orders/admin/all	Admin	Get all orders — filter by ?status=PAID
PATCH	/orders/admin/:id/status	Admin	Manually update any order status
PATCH	/orders/admin/:id/assign-rider	Admin	Assign a rider to a PAID order

6.3 Add to Cart — Request Body

```
{
  "productId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "variantId": "b2c3d4e5-f6a7-8901-bcde-f12345678901", // optional
  "quantity": 2
}
```

6.4 Checkout — Request Body

```
{
  "deliveryAddress": "45 Osu Badu Street, Osu, Accra",
  "deliveryLat": 5.5557,
  "deliveryLng": -0.1969,
  "deliveryNotes": "Call when you arrive, blue gate", // optional
  "deliveryMethod": "EXPRESS" // EXPRESS (GHS 15) | STANDARD (GHS 10)
}
```

6.5 Assign Rider — Request Body (Admin)

```
{
  "riderId": "c3d4e5f6-a7b8-9012-cdef-123456789012"
}
```

7. Payments

Handles MoMo payment initiation via Hubtel, webhook processing, escrow, and wallet payouts. The frontend only needs to call initiate and verify — the webhook is server-to-server.

7.1 Payment Flow

Step 1	Customer checks out — order created with status PENDING
Step 2	Frontend calls POST /payments/initiate with orderId + momoNumber
Step 3	Hubtel sends a payment prompt to the customer's phone
Step 4	Customer approves on their phone
Step 5	Hubtel calls POST /payments/webhook/hubtel (server-to-server, no frontend action)
Step 6	Backend marks transaction SUCCESS + order PAID automatically
Step 7	Frontend polls GET /payments/verify/:reference to check payment status
Step 8	On PAID status — redirect customer to order tracking screen

7.2 Endpoints

Method	Endpoint	Access	Description
POST	/payments/initiate	Customer	Initiate MoMo payment — triggers phone prompt
GET	/payments/verify/:reference	Auth	Poll payment status by internal reference
GET	/payments/transactions	Auth	Get own transaction history
POST	/payments/webhook/hubtel	Hubtel	Hubtel payment callback — do not call from frontend
POST	/payments/payout/vendor	Vendor	Request wallet withdrawal to MoMo
POST	/payments/payout/rider	Rider	Request wallet withdrawal to MoMo
GET	/payments/payouts	Auth	Get own payout history
GET	/payments/admin/transactions	Admin	View all platform transactions
GET	/payments/admin/payouts	Admin	View all platform payouts

7.3 Initiate Payment — Request Body

```
{
  "orderId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "momoNumber": "0551234567"
}
```

7.4 Request Payout — Request Body

```
{
  "amount": 150.00,
}
```

```
"momoNumber": "0241234567"
}
```

7.5 Verify Payment — Response

```
{
  "reference": "KNT-1234567890-ABC123",
  "status":    "SUCCESS",    // PENDING | SUCCESS | FAILED
  "amount":    365.00,
  "orderId":   "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "paidAt":    "2026-03-25T10:30:00.000Z"
}
```

8. Frontend Integration Notes

8.1 Sending Authenticated Requests

Since tokens are stored in httpOnly cookies, the browser sends them automatically. You only need to ensure credentials are included:

```
// fetch
fetch('https://api.kantanova.com/api/v1/auth/me', {
  credentials: 'include'
});

// axios - set globally once
axios.defaults.withCredentials = true;
```

8.2 Handling Token Expiry

The access_token expires after 15 minutes. When any request returns 401, silently call POST /auth/refresh to get a new token pair, then retry the original request.

```
axios.interceptors.response.use(
  response => response,
  async error => {
    if (error.response?.status === 401) {
      await axios.post('/api/v1/auth/refresh');
      return axios(error.config); // retry original
    }
    return Promise.reject(error);
  }
);
```

8.3 Error Response Format

All errors follow this structure:

```
{
  "statusCode": 400,
  "message": "Phone number already registered",
  "error": "Bad Request"
}
```

8.4 Pagination Response Format

All list endpoints return paginated responses in this shape:

```
{
  "data": [ ... ],
  "meta": {
    "total": 120,
    "page": 1,
    "limit": 20,
  }
}
```

```
"totalPages": 6
}
}
```

8.5 Important Notes

Topic	Note
UUID params	All :id params expect a valid UUID — malformed UUIDs return 400
Roles	Hitting an endpoint with the wrong role returns 403 Forbidden
Vendor access	Vendors with PENDING_VERIFICATION status cannot log in until admin verifies
Images	Maximum 5 images per product — upload to Cloudinary first, send URLs
Payment polling	Poll GET /payments/verify/:reference every 5s until status is not PENDING
CORS	Only requests from the registered CLIENT_URL are accepted with credentials
Delivery fee	EXPRESS = GHS 15 STANDARD = GHS 10 — calculated server-side on checkout
Commission	10% platform commission deducted from vendor share on order completion

9. Contact & Support

Backend team	engineering@kantanova.com
Slack channel	#api-integration
Postman collection	Available on request from the backend team
Last updated	March 2026 — v1.0